Exp 13: Case Study on the Limitations of Low-Code Platforms for a Complex Enterprise Use Case

1. Aim

To examine, through a case study, where a low-code platform such as Zoho Creator stops being the right tool when the use case grows into a complex, large-scale enterprise system. The study scales the Gym Membership application built in Experiments 4 to 8 into a national fitness chain, tests each requirement against what low-code can and cannot do, and recommends how the limitations can be handled.

2. Case Background

2.1 The Use Case: A National Fitness Chain

The application built in the earlier experiments serves a single gym. Suppose the same business grows into a fitness chain with several hundred branches and over a million members, and the leadership asks that the existing low-code application be extended to run the whole enterprise. The figures here are assumed for the case study. The requirements are:

- Enrollment, family memberships and fee calculation at every branch (the current application).
- Real-time gate access at peak hours using the member's QR code or a biometric turnstile, with a response time of about a second.
- Month-end billing for the whole member base, with corporate contracts, tiered discounts and automatic renewals.
- Class booking with waiting lists, which is under heavy load during the morning and evening rush.
- A churn-prediction model that flags members likely to leave, built on attendance history.
- Cross-branch analytics for head office, and integration with a legacy ERP and the payment gateways.
- Compliance for members' health data (medical certificates, health declarations) under India's data protection law, with full audit trails.

2.2 Approach

Each requirement was compared with the capabilities of Zoho Creator used in this course (forms, subforms, formula and auto number fields, reports and pages, Deluge workflows, connections, role-based access, environments and Zia). It was then rated as **Fits** (low-code handles it well), **Workaround** (possible with design effort or an add-on service) or **Pro-code needed** (the platform alone is not suitable). The comparison also draws on the trade-offs identified in Exp 1 (vendor lock-in, customization ceiling, data governance and weak testing tools) and the trends in Exp 3.

3. Analysis of Limitations

3.1 Requirement-by-Requirement Assessment

Requirement	What Creator offers	Limitation at enterprise scale	Rating
Enrollment forms, subforms, QR and fee formulas	Form builder, Subform, Formula, Auto Number, QR fields (Exp 4)	Works well. The data model is less flexible than a custom-designed schema, but this is acceptable here.	Fits

24140112

Requirement	What Creator offers	Limitation at enterprise scale	Rating
Pricing with contract rules and tiered discounts	Formula fields and Deluge conditions	Simple rules are easy. A large, changing rule set becomes long Deluge code that is hard to test and maintain.	Workaround
Month-end billing for every member	Deluge loops, scheduled and batch workflows	A Deluge function can execute only a limited number of statements, and loops count every iteration, so one loop over a large member base can be cut off.	Workaround
Real-time gate access at peak	QR field and REST APIs	Shared, multi-tenant infrastructure with execution time limits is not built for sub-second, high-concurrency hardware responses.	Pro-code needed
Churn prediction	Zia and AI fields (Exp 9)	Built-in AI covers common tasks. A custom model trained on the chain's own data has to run outside the platform and be called through an API.	Workaround
Cross-branch analytics	Reports, dashboards and pages (Exp 5)	Reports are built on a single form, so heavy multi-form analysis is better done in a dedicated analytics service.	Workaround
Legacy ERP and turnstile hardware	Connections and API calls (Exp 7)	Ready-made connectors cover mainstream services. Legacy protocols and hardware drivers need middleware or custom code.	Pro-code needed
Health-data compliance and audit	Role-based access, governance and logs (Exp 8, 12)	Compliance is shared with the vendor. Data location and audit depth depend on what the platform exposes, and there is no direct database access.	Workaround
Testing and release	Info logs, Stage and Production (Exp 6, 11)	No mature unit-testing framework or step-through debugger, so regression testing is largely manual.	Workaround
Exit from the platform	REST API and application export	Scripts and structure are tied to the vendor. The exported file holds the application structure and not the records.	Workaround

Table 1: Requirements of the fitness chain assessed against Zoho Creator

3.2 Execution and Scale Limits

The clearest technical ceiling is how much work a single script may do. Zoho documents a cap of roughly 5,000 executed statements per Deluge function (up to about 50,000 depending on the pricing plan), and statements inside a loop are counted once per iteration. A month-end billing loop that updates thousands of member records therefore cannot run as one function. Zoho's suggested remedy is batch workflows, which split the work into chunks. Cloud functions written in Node.js or Java are limited to 40 seconds of execution. These limits suit the form-driven, human-paced work in this course but not heavy batch processing or real-time hardware control.

Limit	Documented value	Effect on the use case
Statements per Deluge function	About 5,000, up to about 50,000 by plan; loops count per iteration	Month-end billing needs batching
Cloud function run time (Node.js, Java)	40 seconds	Long jobs cannot run in one call
Overcoming the statement cap	Batch workflows	Adds design effort and testing

Table 2: Documented platform limits relevant to the case (values vary by plan and may change; check the current Zoho documentation)

3.3 Customization Ceiling and Integration

Visual tools and Deluge cover standard business logic, but turnstile control, biometric devices and a legacy ERP need protocol handling that the platform does not provide natively. As Exp 1 noted, the scripting layer is not a general-purpose language, and built-in connectors serve mainstream services best. For these parts a pro-code extension (for example Zoho Catalyst functions, as explored in Exp 14) or an independent service reached over REST is required.

3.4 Governance, Testing and Vendor Dependence

A chain-wide system holds sensitive health data, so data residency, access control and audit trails become board-level concerns. Creator offers role-based access, environments and logs, but the organisation shares the compliance burden with the vendor and can audit only what the platform exposes. Testing is also weaker than in traditional development: there is no unit-test framework or debugger, and the info logs used in Exp 6 are the main tool. Finally, the application lives inside the vendor's ecosystem. A practical illustration came during this lab: when the free trial expired, the application could not be exported before access was lost, which showed how dependent the work is on the account's subscription status.

4. Recommended Approach

The analysis does not point to abandoning low-code. It points to using it where it is strong and using code where it is not: a hybrid architecture, as suggested by the fusion-team model in Exp 3. Table 3 shows a sensible split.

Layer	Built with	Reason
Branch front desk: enrollment, families, fees, reports	Zoho Creator (low-code)	Fast to build, easy for staff to change, already working
Class booking and approvals	Zoho Creator with batch workflows	Standard workflow logic, scaled by batching
Gate-access service	Pro-code service (for example Catalyst functions) reached through REST	Needs low latency and hardware integration
Month-end billing engine	Pro-code batch service writing back through the Creator API	Avoids statement and run-time ceilings
Churn model and analytics	External ML service and an analytics platform	Custom models and multi-form analysis
Governance	Centre of Excellence, role-based access, environments, regular exports	Controls risk and keeps an exit path open

Table 3: Recommended split between low-code and pro-code for the fitness chain

5. Observations

- Low-code fits the single-branch and front-desk workloads very well, which is why the gym application could be built within the course.
- The limits appear when volume, speed or protocol demands rise: batch billing, real-time gate access and legacy integration.
- Most limitations have a workaround, but each workaround adds code, testing effort or another service, which reduces the original speed advantage.
- Vendor lock-in, shared compliance responsibility and weak testing tools matter more as the application becomes business-critical.

24AIO113

- A hybrid design keeps low-code for what it does best and confines pro-code to the few components that need it.

6. Conclusion

For a complex enterprise use case such as a national fitness chain, a low-code platform like Zoho Creator is a strong base for forms, workflows, reporting and internal tools, but it is not suitable as the only technology. Statement and run-time limits, the customization ceiling, shared compliance duties, limited testing tools and vendor dependence make it necessary to move performance-critical and hardware-facing parts to pro-code services. Choosing the platform therefore depends on matching each requirement to the right layer, and not on whether low-code is “better” in general.

References

Zoho Creator Help: “Workflows – Limitations” (help.zoho.com) and “Deluge Limitations” (zoho.com/creator/help/script/usage-limits.html). Platform limits are quoted as documented at the time of writing and vary by plan.

2414011142